

A Cloud-Native Multi-Agent Orchestration Framework for Autonomous Utility Services

Abstract

Purpose: Traditional AI deployments in customer service remain reactive and lack multi-agent coordination. We present the Universal Utility Orchestrator (UVO), a cloud-native framework enabling autonomous, extensible workflows through a centralised LLM reasoning engine and a lifecycle-managed agent registry.

Methodology: We conducted a proof-of-concept feasibility study evaluating UVO over 180 utility task runs based on a reference dataset of two customer accounts. System performance was compared against one-shot LLM and retrieval-augmented generation (RAG) baselines.

Findings: UVO achieved a task success rate of 0.98 (vs. 0.82 one-shot LLM, 0.85 RAG) and fact coverage of 0.96 (vs. 0.72, 0.80), alongside an $8.4\times$ latency increase. UVO also demonstrated strong agent-selection F1 (0.80) and sequence accuracy (0.98), metrics structurally inapplicable to single-agent baselines.

Conclusion: Dynamic orchestration acts as a critical missing layer in agentic architectures, offering a scalable foundation for deploying autonomous systems in complex utility environments.

1 Introduction

The growing adoption of artificial intelligence in customer-facing systems has yielded measurable improvements in accessibility and operational efficiency. Despite this progress, most deployed systems remain fundamentally reactive: they answer predefined queries or execute narrowly scoped tasks, but cannot autonomously compose multi-step workflows in response to evolving context [2, 15]. In the utilities sector, where grid telemetry, billing records, weather feeds, and outage logs are continuously generated, the inability to reason across these heterogeneous streams limits the practical value of AI to relatively shallow question-answering.

Recent advances in LLM-based agentic systems have introduced the possibility of more autonomous AI behaviour. Frameworks such as ReAct [3], AutoGPT [5], and LangGraph [6] demonstrate that LLMs can perform multi-step reasoning, invoke external tools, and adapt to diverse objectives without explicit procedural programming. Similarly, retrieval-augmented gener-

ation (RAG) [7] has shown that grounding LLM responses in external knowledge substantially reduces hallucination and improves factual accuracy. Nevertheless, these systems are typically evaluated in single-agent settings, and their design does not address the coordination, lifecycle management, and observability challenges that arise when multiple specialised agents must collaborate dynamically.

We argue that the primary limitation of current agent-based systems is not individual agent capability, but the absence of a unified *orchestration layer* capable of coordinating, sequencing, and managing interactions among multiple specialists. Without such a layer, constructing complex multi-agent workflows requires significant manual engineering, tightly coupled pipelines, or task-specific implementations that resist generalisation [8].

To address this gap, we propose the **Universal Utility Orchestrator (UVO)**, a modular, cloud-native orchestration framework for heterogeneous agent workflows. The system introduces a centralised orchestrator that leverages LLM-based reasoning to dynamically generate execution plans, select appropriate agents from a managed registry, and aggregate results into coherent responses. This design is evaluated in the domain of utility customer service, a setting characterised by diverse query types, structured operational data, and clear ground-truth expectations.

The main contributions of this paper are:

1. **Orchestration architecture:** A centralised reasoning layer that dynamically plans and executes multi-agent workflows without hard-coded pipelines.
2. **Plug-and-play agent registry:** A lifecycle-managed registry that enables agents to be added, removed, or updated at runtime without modifying orchestrator logic.
3. **Standardised communication protocol:** A uniform request-response contract that decouples agent implementation from orchestration concerns.
4. **Empirical feasibility study:** A 180-run comparative study against one-shot LLM and RAG baselines across six task categories, with analysis of quality, efficiency, and robustness metrics.

The remainder of the paper is structured as follows. Section 2 reviews related work. Section 3 provides a high-level system overview. Section 4 describes the core architectural components. Section 5 details the execution model. Section 6 presents use cases and test scenarios. Section 7 reports the evaluation. Section 8 discusses implications and limitations. Section 9 concludes.

2 Related Work

The proposed framework synthesises contributions from four lines of prior research: multi-agent systems, LLM-based agentic frameworks, workflow orchestration systems, and cloud-native microservice architectures.

2.1 Multi-Agent Systems

Multi-agent systems (MAS) have been studied extensively as a paradigm for distributed problem-solving [9]. Classical MAS research addresses coordination protocols, communication strategies, and task decomposition, and provides strong theoretical foundations for agent interaction. However, traditional MAS approaches typically rely on predefined ontologies and symbolic coordination rules, limiting their adaptability in unstructured, data-driven environments [10]. Recent surveys confirm that dynamic, LLM-driven coordination represents an open challenge in the field [2].

2.2 LLM-Based Agentic Frameworks

The emergence of instruction-following LLMs has catalysed a new class of agentic frameworks. ReAct [3] interleaves reasoning traces with tool-use actions in a single model, demonstrating that LLMs can plan and act without explicit procedural programming. Reflexion [1] extends this by grounding agent self-improvement in verbal feedback rather than gradient updates. AutoGPT [5] and BabyAGI [16] popularised the idea of open-ended autonomous agents that decompose high-level goals into subtasks, while LangGraph [6] and similar graph-based frameworks provide structured execution flows for LLM pipelines. AgentBench [11] offers a systematic benchmark for evaluating LLM agents across multiple environments. A common limitation of these systems is their focus on single-agent capability: they provide limited support for managing multiple agents simultaneously, with little attention to agent lifecycle, inter-agent coordination, or runtime extensibility.

2.3 Workflow Orchestration and Pipelines

Workflow orchestration systems such as Apache Airflow [12], Prefect [17], and Temporal [18] are widely deployed in data engineering to coordinate complex

processes across distributed services. These systems provide reliability and observability through directed acyclic graph (DAG) execution models. Nonetheless, their workflows are largely static: execution graphs are defined at authoring time and cannot adapt dynamically to query content or runtime context [8]. Our framework adopts a *dynamic* orchestration strategy in which the execution plan is synthesised at query time by an LLM planner, removing the dependency on pre-authored graphs.

2.4 Cloud-Native and Microservice Architectures

Microservices and cloud-native design patterns [13] have enabled scalable, loosely coupled systems through containerisation, serverless computing, and API-based communication. Kubernetes [14] and related platforms provide infrastructure abstractions that decouple application logic from deployment concerns. While these approaches support modularity and horizontal scalability, they do not address intelligent coordination between services. Our registry-centric design adopts microservice principles to manage agent lifecycle while adding LLM-driven coordination on top of the infrastructure layer.

2.5 Positioning

Collectively, prior work has advanced individual agent capability, retrieval quality, and distributed execution infrastructure. What is absent is an architectural layer that *coordinates* these components: selecting the right agents, sequencing their execution, managing their lifecycle, and aggregating their outputs. UVO addresses this gap by treating orchestration as a first-class design concern rather than a secondary engineering task.

3 System Overview

The Universal Utility Orchestrator is designed around a single core principle: the orchestrator should be the sole component that knows *what* needs to be done; individual agents should only need to know *how* to do their specific task. This separation of concerns enables the system to scale across domains without modifying agent implementations.

At the highest level, UVO comprises three interacting subsystems:

1. **Orchestrator:** A stateful reasoning engine that receives user queries, retrieves the current agent registry, generates an execution plan via an LLM planner, dispatches calls to selected agents, and synthesises a final response.
2. **Agent Registry:** A managed catalogue of available specialist agents, each described by a metadata

record that includes its capabilities, input/output schema, and current lifecycle state.

3. **Specialist Agents:** Stateless, single-purpose services that accept standardised requests from the orchestrator and return structured results.

The framework is guided by three design principles:

Modularity. Agents are independently deployable units with no knowledge of each other. Adding a new capability requires registering a new agent; no changes to existing components are needed.

Scalability. All components are containerised and stateless where possible, enabling horizontal scaling. The registry is the only shared state, accessed through an idempotent API.

Observability. Every orchestration cycle emits a structured trace recording the planner’s output, each agent’s inputs and outputs, timing data, and the final synthesised response. This enables post-hoc auditing, debugging, and evaluation.

Figure 1 illustrates the high-level component topology.

4 Architecture Design

4.1 Orchestrator Layer

The orchestrator layer is the decision-making core of UVO. Its primary responsibility is to translate a natural-language user query into a dependency-aware execution plan over registered specialist agents and to synthesise the resulting outputs into a coherent, grounded response.

Internal components. The orchestrator contains two LLM-driven sub-components and a deterministic execution engine:

- **Planner:** Given the user query, conversation transcript, and a serialised manifest of active registry agents (name, description, and capability list), the planner produces a structured execution plan — a JSON object enumerating the agents to invoke, per-step task instructions, explicit `depends_on` edges, and a `synthesis_instruction` for the final answer. The planner prompt is system-injected and instructs the model to reason about data dependencies (e.g., customer lookup must precede billing retrieval) before committing to a directed acyclic graph. Plans with cyclic dependencies are rejected.
- **Executor:** After planning, the executor validates the DAG, dispatches agent calls in topological order, and runs independent steps concurrently when their dependencies are satisfied. It applies per-agent invocation configuration (HTTP method, body template, or schema-driven field mapping), injects authentication credentials, enforces timeouts and retries, and records per-step results and errors.
- **Synthesiser:** After all agents have responded, the synthesiser receives the execution trace, prior conversation transcript, and `synthesis_instruction`, and produces a natural-language response grounded in the retrieved facts. Its prompt instructs it to cite only data present in agent outputs or the transcript, suppressing hallucination.

Execution responsibilities. The orchestrator manages the complete lifecycle of a query: input guardrails for reactive requests, session and transcript persistence, plan generation, agent dispatch, timeout and retry handling, response synthesis, and trace commitment. For follow-up turns where prior transcript already contains sufficient information, the planner may return an empty step list and delegate answering entirely to the synthesiser.

Design rationale. Centralising planning in a single component ensures that data-dependency reasoning occurs once, at plan-generation time, rather than being distributed across agents. Separating execution into a deterministic executor keeps agent invocation, authentication, and failure handling auditable and independent of LLM variability. This simplifies individual agent implementations and makes the system’s reasoning behaviour inspectable through the persisted plan record.

4.2 Agent Registry

The agent registry is a metadata service that maintains the catalogue of available specialist agents and serves as the integration boundary for new capabilities.

Data model. Each registry entry contains: a unique agent name, a natural-language description, a registered `endpoint_url`, lifecycle status (`active` | `inactive` | `degraded`), version metadata, optional capability records (each with its own input/output schema), and three integration-critical configuration blocks: `invocation_config` (HTTP method, request schema or body template, response extraction path), `auth_config` (API key, bearer token, OAuth, or custom credential injection), and `health_check_config`. Registry documents are persisted in Cosmos DB and shared by both the registry admin API and the orchestrator runtime.

Lifecycle operations. The registry exposes a RESTful API supporting core CRUD operations: `GET /api/agents` (list/filter), `POST /api/agents` (register), `PATCH /api/agents/{id}` (update metadata or configuration), and `DELETE /api/agents/{id}` (soft deactivation by default). Additional endpoints support health probing, capability indexing, export, and trace inspection. At the start of each query cycle, the orchestrator queries Cosmos DB for all `active` agents, so a status change or new registration takes effect on the very next query without an orchestrator restart.

Design rationale. By making the registry the sole integration point, UVO ensures that the question

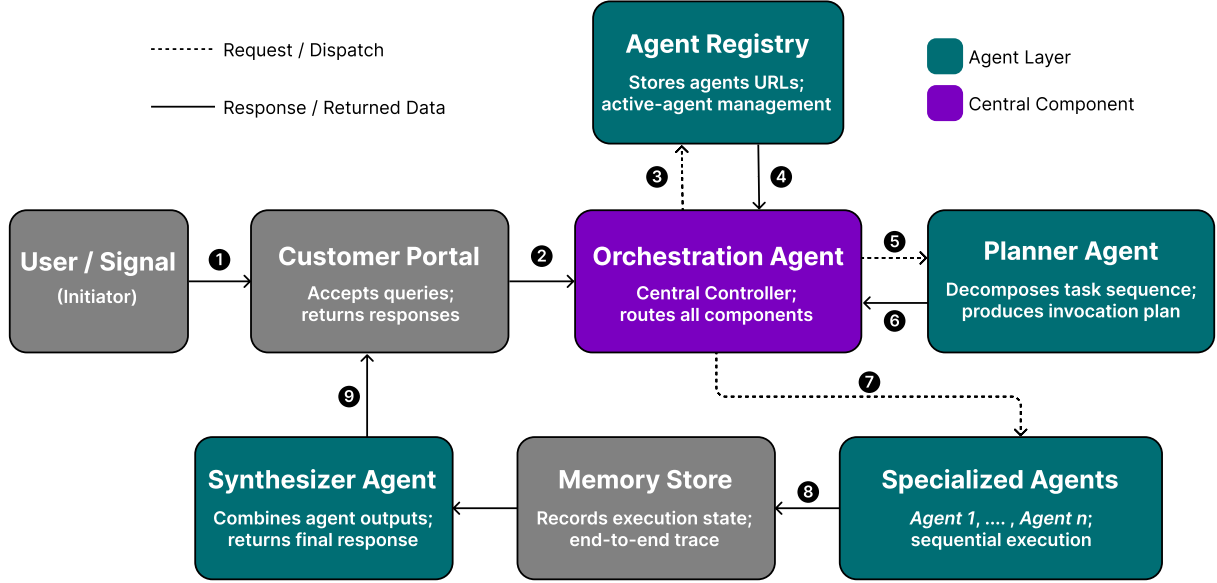


Figure 1: High-level topology of the Universal Utility Orchestrator. User queries enter the orchestrator, which consults the agent registry, generates a plan, dispatches to specialist agents, and returns a synthesised response.

“what can the system do?” has a single, authoritative answer. The plug-and-play property demonstrated in Section 7 follows directly from this design: adding a new specialist is primarily a registry configuration action rather than an orchestrator code change.

4.3 Communication Layer

The communication layer defines the interaction contracts between the orchestrator and individual agents.

Protocol. All orchestrator-to-agent communication uses HTTP/JSON. Each agent registers a concrete endpoint URL and route (e.g. `POST /api/billing_agent`); there is no single universal invoke path. The orchestrator constructs request bodies in one of three modes configured per agent: a default `AgentRequest` envelope containing `task`, `session_id`, `customer_id`, `context` (outputs from prior steps), and `conversation_history`; a template-driven body with token substitution; or a schema-driven mapper that populates fields from the plan step and prior outputs. Responses are parsed via a configurable result path; the canonical shape includes `agent_name`, `result`, and optional `actions_taken`, `suggestions`, and `metadata`.

Synchronous vs. asynchronous handling. Individual agents respond synchronously to each HTTP request. The orchestrator may dispatch independent plan steps concurrently within a dependency layer, collecting results before proceeding to dependent steps. This distinction is abstracted from agents, which always see a single request/response interaction.

Standardisation rationale. Although agents expose heterogeneous endpoint paths, the registry’s `invocation_config` and `auth_config` provide a uni-

form integration surface for the orchestrator. Any HTTP service that conforms to its registered contract can be invoked without modification to orchestrator logic.

4.4 Observability and Logging

End-to-end observability is a first-class design requirement in UUO, motivated by the need for post-hoc auditing in regulated domains and for evaluation support.

What is logged. For every query cycle, the system emits a structured trace document containing: the raw user query; the plan produced by the planner (intent, step count, synthesis instruction); per-step records including `agent_name`, request payload, raw response, extracted result, dependency metadata, schema-mapping details, and wall-clock duration; optional stage events for demo and streaming modes; the synthesiser’s final response; overall latency; and error state. Traces are persisted in Cosmos DB as one document per session, updated incrementally as execution progresses, with a configurable time-to-live.

Usage. Execution traces serve three purposes: (1) operational debugging, enabling engineers to identify which step in a multi-agent chain produced an incorrect intermediate result; (2) evaluation, as the 180-run study in Section 7 derives all metrics directly from trace records; and (3) audit compliance, providing a complete record of data accessed and decisions made for a given query. The registry exposes trace-list and trace-detail APIs for inspection in the admin UI.

4.5 Cloud-Native Deployment

UUO is designed for containerised, cloud-native deployment on managed PaaS infrastructure, with modular infrastructure-as-code for repeatable environment provisioning.

Infrastructure. The control plane — orchestrator and agent registry — is deployed as independent Azure Function Apps, provisioned via reusable Terraform modules (resource group, Cosmos DB, Key Vault, OpenAI, and function-app modules). Each specialist agent is packaged as an independent Docker image and deployed to Azure Container Apps, enabling per-agent lifecycle management without coupling to the orchestrator release cycle. For local development and demos, the full backend stack — orchestrator, registry, Cosmos emulator, and demo frontends — runs via Docker Compose, preserving environment parity through identical container images. Terraform parameterisation (`environment`, `suffix`, `region`) supports redeployment across dev, staging, and production within Azure.

Execution model. The system exposes orchestration through HTTP endpoints on the orchestrator Function App, principally `POST /api/chat` for synchronous multi-agent queries and `POST /api/chat/stream` for server-sent-event streaming. The registry Function App exposes a separate admin API (`/api/agents`) for agent CRUD, health probes, and trace inspection. Agent invocation is decoupled: the orchestrator resolves registered agent endpoints from Cosmos DB and calls them over HTTPS, so any HTTP-compliant service can be plugged in without redeploying the control plane. In production, Managed Identity and Key Vault enforce zero-trust credential handling; session state, execution traces, and registry metadata persist in Cosmos DB.

Benefits. This architecture supports independent scaling of the serverless control plane (consumption-based Function Apps) and per-agent Container App replicas, rolling agent updates via container image replacement, and environment parity between local Docker Compose and cloud deployment. Modular Terraform enables the control-plane footprint to be reproduced across environments; specialist agents remain independently versioned and deployable.

5 Execution Model

5.1 High-Level Pipeline

A complete query cycle traverses the following stages:

1. **Input processing:** The orchestrator receives and validates the user query.
2. **Planning:** The planner LLM is invoked with the query and the serialised active-agent registry. It produces a JSON execution plan.

3. **Agent selection:** The orchestrator parses the plan, validates agent identifiers against the registry, and constructs the invocation queue.
4. **Execution:** Agents are invoked in plan order. Results from earlier steps are injected as context into subsequent calls where specified by the plan.
5. **Aggregation:** All agent responses are collected into a structured results bundle.
6. **Synthesis:** The synthesiser LLM receives the results bundle and generates the final natural-language response.
7. **Logging:** The full trace is written to the log store.
8. **Response:** The synthesised answer is returned to the caller.

5.2 State Transitions

The orchestrator implements a lightweight state machine governing each query cycle:

Idle → Planning → Executing → Waiting →
Aggregating → Logging → Idle

The **Waiting** state is entered when an asynchronous agent call is outstanding. On timeout, the orchestrator transitions to **Aggregating** with a partial result, allowing synthesis to proceed with available data.

Algorithm 1 provides a pseudo-code summary.

Algorithm 1 UUO Query Execution Loop

Require: User query q , agent registry $\mathcal{R}$

Ensure: Response r , trace τ

```

1:  $P \leftarrow \text{Planner}(q, \mathcal{R})$     ▷ Generate DAG execution plan
2:  $\text{results} \leftarrow \{\}; \tau \leftarrow []$ 
3:  $\text{queue} \leftarrow \text{TopologicalSort}(P)$ 
4: while queue is not empty do
5:    $\text{batch} \leftarrow \text{GetIndependentSteps}(\text{queue}, \text{results})$ 
6:   for each  $s \in \text{batch}$  in parallel do
7:      $a \leftarrow \mathcal{R}[s.\text{agent\_id}]$     ▷ Look up agent
8:      $\text{ctx} \leftarrow \text{BuildContext}(s, \text{results})$ 
9:      $\text{resp}_s \leftarrow a.\text{invoke}(\text{ctx})$     ▷ Call agent
10:     $\text{results}[s.\text{agent\_id}] \leftarrow \text{resp}_s$ 
11:     $\tau.\text{append}(\text{TraceRecord}(s, \text{resp}_s))$ 
12:   end for
13: end while
14:  $r \leftarrow \text{Synthesiser}(q, \text{results})$ 
15:  $\tau.\text{append}(\text{FinalRecord}(r))$ 
16: return  $r, \tau$ 
```

6 Use Cases and Evaluation Scenarios

We evaluate UUO in the domain of utility customer service, focusing on a reference customer dataset cover-

ing two accounts (CUST-1001, CUST-1002) over a July 2019 billing window. Task scenarios are grouped into two classes.

6.1 Reactive Workflows

Reactive workflows are triggered by an explicit user query. Six task categories were defined:

- **Billing:** Explain a high bill, break down charges (e.g., “Why was CUST-1001 charged so much for July 2019?”).
- **Anomaly detection:** Identify usage spikes or irregular consumption patterns.
- **Outage:** Report on service interruptions and their duration.
- **Payment risk:** Assess customer payment history and flag risk.
- **Solar:** Evaluate solar panel performance and credit calculations.
- **Weather correlation:** Determine whether usage anomalies correlate with weather events.

6.2 Proactive Workflows and Multi-Hop Queries

Five of the 30 tasks are classified as multi-hop, requiring the orchestrator to compose five or more agents in sequence. An example is: “Are the spikes in CUST-1001’s July 2019 usage tied to the weather?” — which requires customer lookup, usage retrieval, anomaly detection, weather data retrieval, and correlation analysis. These tasks test the orchestrator’s ability to propagate intermediate results as context across agent boundaries.

Each task is expressed in three paraphrase variants to assess robustness (e.g., “Why was my bill high?”, “My July bill spiked”, “What caused the extra charges?”).

7 Evaluation

7.1 Evaluation Objectives

The evaluation addresses four questions:

1. Does orchestration improve task quality over flat LLM and RAG baselines?
2. Does the orchestrator correctly identify and sequence specialist agents?
3. How does orchestration affect system latency?
4. How robustly does the system handle paraphrase variation?

7.2 Experimental Setup

Systems compared. Three systems are evaluated: (i) *One-shot LLM*: a direct call to the base LLM with no tools or retrieval; (ii) *RAG*: the same LLM augmented with retrieval over the full data corpus used by the orchestrator’s agents; (iii) *UUO*: the full orchestration system.

Task set. 30 tasks across six categories, each expressed in two runs per (task, system) pair, for a total of 180 LLM calls.

Ground truth. Each task in `tasks.json` specifies: `expected_facts` (substrings that must appear in a correct answer), `expected_agents` (the set of agents a correct orchestration should invoke), and `expected_sequence` (the correct invocation order).

7.3 Metrics

Task success rate (binary 0/1): A response is judged successful if fact coverage ≥ 0.5 and no “punt” phrase (e.g., “*I don’t have access to that data*”) is present, or if fact coverage ≥ 0.7 regardless of punt phrases. Reported as the mean across 60 runs per system.

Agent F1: Standard precision/recall F1 over the agent set, comparing `agents_used` against `expected_agents`. Order is ignored; sequence accuracy handles ordering separately. Structurally 0 for baselines, which invoke no agents.

Sequence accuracy: Longest-common-subsequence (LCS) ratio between the expected agent sequence and the actual invocation order derived from trace timestamps. Because the expected sequences in our ground truth were authored by the system designers, this metric primarily measures the planner’s consistency with design intent rather than an independently validated ground truth. It rewards similar ordering even when extra agents are inserted.

Fact coverage: Fraction of `expected_facts` substrings present in the final response (case-insensitive). A literal \$ token acts as a wildcard matching any dollar amount.

Average latency: End-to-end wall-clock time measured around the HTTP call from the evaluation harness, reported as the mean over 60 runs.

7.4 Results

7.4.1 Headline Metrics

Table 1 reports aggregate results across all three systems. UUO outperforms both baselines on every quality metric. Task success improves from 0.85 (RAG) to 0.98 (+15%), fact coverage from 0.80 to 0.96 (+20%), while achieving non-trivial agent selection (F1 = 0.80) and near-perfect sequence fidelity (0.98) — dimensions entirely absent in the baselines. This quality gain comes at the cost of an approximately 8.4× increase in end-to-end latency (25.1 s vs. 3.0 s for the one-shot baseline).

Table 1: Aggregate evaluation metrics (180 runs, 60 per system). Best values in **bold**.

System	Success	Agent F1	Seq. Acc.	Fact Cov.	Latency
One-shot LLM	0.82	0.00	0.00	0.72	3.0 s
RAG	0.85	0.00	0.00	0.80	3.7 s
UO	0.98	0.80	0.98	0.96	25.1 s

7.4.2 Fact Coverage by Category

Table 2 disaggregates fact coverage by task category.

Table 2: Fact coverage by task category. “Gap” is the orchestrator minus the best baseline.

Category	One-shot	RAG	UO	Gap
Billing	0.47	0.85	1.00	+0.15
Anomaly	0.56	0.71	0.96	+0.25
Outage	0.80	0.80	0.90	+0.10
Payment risk	0.75	0.90	0.90	0.00
Solar	1.00	1.00	1.00	0.00
Weather	0.85	0.57	1.00	+0.15

The orchestrator’s advantage is largest on anomaly (+0.25), billing (+0.15), and weather (+0.15). The weather result is architecturally significant: RAG’s score drops to 0.57 because weather data resides in a live agent rather than the static retrieval corpus, confirming that structured agent dispatch is necessary for tasks requiring dynamic or external data.

7.4.3 Latency Breakdown

Table 3 decomposes the orchestrator’s wall-clock time.

Table 3: Orchestrator latency breakdown.

Phase	Mean	Share
Reasoning (planner + synthesiser)	16.5 s	66%
Agent execution (summed steps)	8.6 s	34%
Total wall-clock	25.1 s	100%

Two-thirds of latency is attributable to LLM reasoning, not agent execution. This identifies the optimisation target unambiguously: future work should focus on reducing planner and synthesiser latency (e.g., through smaller specialised models or prompt caching) rather than on agent-side optimisations.

7.4.4 Robustness to Rephrasing

Within paraphrase groups, the orchestrator selects the same agents in 96% of runs. Baseline consistency is trivially 1.00 (no agents are selected), so this metric is meaningful only for UO. A 4% inconsistency rate on agent selection across phrasings represents a minor robustness gap and is discussed further in Section 8.

7.4.5 Plug-and-Play Demonstration

To demonstrate the operational value of the registry design, we issued a solar query before and after deregistering the `solar_performance_credit_loss_agent` via a single `PATCH` API call. No orchestrator, planner, or synthesiser code was modified between runs. The planner incorporated the newly-active agent on the very next query and arrived at the opposite — and correct — conclusion (solar over-performance of 7.2%; not the cause of the high bill), compared to the incorrect inference drawn from missing invoice fields in the agent’s absence. This demonstrates that the registry is a true integration boundary: extending system capability is a configuration action, not a code change.

7.5 Caveats

Several methodological limitations should be noted. First, task success is judged automatically by `judge_success.py` rather than human raters; baseline success rates (0.82, 0.85) may be overstated, as the judge rewards vague but keyword-matching responses. Second, orchestrator token usage was not fully instrumented; we estimate 3–5× the baselines’ 300–1800 tokens per query, but do not report this formally. Third, the solar category exhibits loose fact coverage due to near-universal presence of trigger substrings, making all three systems trivially pass. Fourth, all tasks reference `CUST-1001/CUST-1002` in July 2019; generalisation beyond this window is outside the scope of the current evaluation. Fifth, the ground-truth rubrics (`expected_agents` and `expected_sequence` in `tasks.json`) were authored by the same team that designed the agent registry. The sequence accuracy result of 0.98 therefore partially measures how well the planner reproduces the authors’ own orchestration intuitions rather than an independently validated ground truth.

8 Discussion

8.1 Key Insights

Orchestration earns its complexity on compositional tasks. On single-agent tasks (outage, payment risk), RAG narrows the performance gap substantially because flat retrieval is sufficient. On tasks requiring multiple structured sources (billing, anomaly, weather, multi-hop), the orchestrator’s advantage widens. This

confirms our core hypothesis: orchestration is most valuable precisely where isolated agents or retrieval systems are least adequate.

The latency bottleneck is reasoning, not execution. The $8.4\times$ latency increase relative to baselines is dominated by the LLM planning and synthesis phases (66% of total latency), not agent execution. This has a clear architectural implication: optimising agent backends will have limited impact. More promising approaches include prompt caching for the planner, parallelising independent agent calls, or replacing the general-purpose planner with a smaller, fine-tuned routing model.

RAG with the same data still under-performs orchestration. Even when provided access to the same data corpus used by the orchestrator’s agents, RAG achieves lower fact coverage (0.80 vs. 0.96) and fails on multi-hop queries that require propagating intermediate results. Structured orchestration is performing work beyond what flat retrieval can achieve.

Failure mode analysis. With a task success rate of 0.98 over 60 runs, the orchestrator produced approximately one task failure and one ordering error. However, the agent selection F1 score of 0.80 indicates meaningful divergence from the expected agent sets. Analysis reveals this is driven almost entirely by false positives (the orchestrator calling additional unnecessary agents) rather than false negatives (missing the required agents). Because the planner is prompted to prioritize comprehensiveness, it frequently invokes supplementary agents (e.g., invoking the `weather_context_agent` even when a billing anomaly is cleanly explained by a rate change). While this defensive planning ensures high fact coverage (0.96), it suppresses the precision component of the F1 score and incurs unnecessary latency. In the singular observed task failure, the planner omitted the `weather_context_agent` (a false negative) when the query expressed the weather connection only implicitly (e.g., “*What caused the July spike?*”), causing an incomplete conclusion. Both the F1 precision loss and this specific failure point to the same root cause: planner sensitivity to implicit versus explicit query framing, motivating future work on prompt refinement.

8.2 Trade-offs

The primary trade-off in UVO is between response quality and latency. At 25.1 seconds mean latency, the system is unsuitable for latency-sensitive applications such as real-time chat interfaces, but is well-suited for asynchronous workflows, background analysis tasks, or operator-facing dashboards. A configurable quality-latency operating point (e.g., limiting plan depth for simple queries) is a practical design direction. Cost is a related consideration: although token usage was not fully instrumented, the planner and synthesiser together are estimated to consume $3\text{--}5\times$ the baselines’

300–1 800 tokens per query, implying a per-query cost of roughly \$0.01–\$0.05 at current API pricing — acceptable for asynchronous or operator-facing workloads, but a relevant constraint for high-volume consumer deployments.

8.3 Limitations

Beyond the evaluation caveats noted above, UVO has several architectural and methodological limitations. The centralised orchestrator is a single point of failure in the current reference implementation, though this is addressable through standard high-availability patterns. The system’s performance is sensitive to planner prompt quality: poorly described agent capabilities in the registry can lead to incorrect agent selection. Additionally, the 4% agent-selection inconsistency under rephrasing suggests that the planner is affected by natural language queries and warrants further study.

Methodologically, the most critical limitation is the narrow scope of the evaluation dataset, which rests on two customer accounts over a single billing month. Consequently, the findings should be interpreted as a proof-of-concept feasibility study rather than a generalised benchmark of the systems’ utility domain performance. Furthermore, the evaluated baselines (one-shot LLM and RAG) are single-agent systems. A comprehensive evaluation of UVO must ultimately include comparisons against existing multi-agent frameworks, such as AutoGen or LangGraph, to isolate the specific advantages of our registry-based orchestration pattern. We defer this comparative analysis to future work.

8.4 Broader Implications

We believe orchestration should be elevated to a first-class design concern in agentic system architectures, on par with individual agent capability and retrieval quality. The results demonstrate that even with a modest set of specialised agents, dynamic orchestration substantially outperforms monolithic LLM approaches on the tasks where it matters most. More broadly, the registry-based design pattern offers a reusable blueprint for any domain in which capabilities must be added incrementally without system-wide redeployment.

9 Conclusion

We presented the Universal Utility Orchestrator, a cloud-native multi-agent orchestration framework that addresses the coordination gap in current agentic AI architectures. By centralising dynamic planning in an LLM-driven orchestrator layer, managing agent capabilities through a lifecycle-aware registry, and enforcing standardised communication contracts, UVO enables scalable, composable, and observable multi-agent workflows without tight coupling or manual pipeline engineering.

An empirical evaluation across 180 task runs demonstrated that UUO achieves a task success rate of 0.98 and fact coverage of 0.96 — significantly outperforming one-shot LLM (0.82 / 0.72) and RAG (0.85 / 0.80) baselines — while maintaining near-perfect agent sequence fidelity (0.98) and 96% robustness to query rephrasing. The primary cost is an $8.4\times$ latency increase, traced principally to LLM reasoning rather than agent execution.

Future Work

Several directions merit investigation. First, **planner efficiency**: replacing the general-purpose planner with a fine-tuned routing model or leveraging prompt caching could substantially reduce latency without sacrificing plan quality. Second, **parallel execution**: the current sequential execution model could be replaced with dependency-graph-based parallelism for independent agent steps. Third, **broadier domain evaluation**: extending the evaluation beyond the July 2019 utility window and to other sectors (e.g., healthcare, finance) would test generalisation of the orchestration approach. Fourth, **human evaluation**: complementing automated judges with human raters would yield more reliable task success estimates, particularly for the baselines. Fifth, **token efficiency**: full instrumentation of token usage across all system components would enable a complete cost-quality analysis.

Statements and Declarations

Ethical considerations

Not applicable.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Declaration of conflicting interest

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding statement

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Data availability

Not applicable.

References

- [1] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [2] L. Wang et al., “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [3] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [4] Q. Wu et al., “AutoGen: Enabling next-generation LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [5] T. Significant Gravitas, “AutoGPT: An autonomous GPT-4 experiment,” GitHub repository, 2023. [Online]. Available: <https://github.com/Significant-Gravitas/AutoGPT>
- [6] LangChain, Inc., “LangGraph: Build stateful, multi-actor applications with LLMs,” 2024. [Online]. Available: <https://langchain-ai.github.io/langgraph/>
- [7] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [8] Z. Xi et al., “The rise and potential of large language model based agents: A survey,” *arXiv preprint arXiv:2309.07864*, 2023.
- [9] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, UK: Wiley, 2009.
- [10] N. R. Jennings, K. Sycara, and M. Wooldridge, “A roadmap of agent research and development,” *Autonomous Agents and Multi-Agent Systems*, vol. 1, no. 1, pp. 7–38, 1998.
- [11] X. Liu et al., “AgentBench: Evaluating LLMs as agents,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [12] Apache Software Foundation, “Apache Airflow,” 2015. [Online]. Available: <https://airflow.apache.org>
- [13] M. Fowler and J. Lewis, “Microservices,” *martinfowler.com*, 2014. [Online]. Available: <https://martinfowler.com/articles/microservices.html>
- [14] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016.

- [15] P. Maes, “Agents that reduce work and information overload,” *Communications of the ACM*, vol. 37, no. 7, pp. 30–40, 1994.
- [16] Y. Nakajima, “BabyAGI,” GitHub repository, 2023. [Online]. Available: <https://github.com/yoheinakajima/babyagi>
- [17] Prefect Technologies, Inc., “Prefect: The new standard in dataflow automation,” 2018. [Online]. Available: <https://www.prefect.io>
- [18] Temporal Technologies, Inc., “Temporal: Build invincible apps,” 2020. [Online]. Available: <https://temporal.io>